

Foundry-lite AIP 도입을 위한 심층 아키텍처 분석 및 실행 설계서

Palantir AIP 공개 아키텍처와 foundry-lite main 코드의 교차검증

Foundry-lite Architecture Review

2026-06-25 | Version 1.0

문서 성격. 본 문서는 Palantir 가 공개한 AIP·Foundry·Apollo 문서와 ludia8888/foundry-lite 의 main 코드 및 구현 상태 문서를 교차검증하여 작성한 기술 설계서다. Palantir 내부 소스 코드, 비공개 데이터베이스 제품, 내부 메시지 버스 및 서비스 메시 구현은 확인할 수 없으므로, 공개적으로 확인되는 제품 계약과 동작을 기준으로 재구성한다. 추정이 포함된 부분은 명시적으로 “아키텍처 추론” 또는 “권고안”으로 구분한다.

문서 구성

1. 경영·아키텍처 요약
2. 조사 범위와 판독 원칙
3. Palantir AIP 작동원리의 복원
4. foundry-lite 현행 아키텍처 진단
5. AIP와 foundry-lite의 기능·구조 교차대조
6. 핵심 갭과 우선 위험
7. 목표 AIP-lite 참조 아키텍처
8. 핵심 컴포넌트 상세 설계
9. 보안·권한·데이터 반출 통제
10. 데이터 모델과 상태 머신
11. API·포트·패키지 구조
12. 종단간 실행 시퀀스
13. 첫 번째 제품 수직 슬라이스
14. 구현 로드맵과 PR 분해
15. 테스트·평가·릴리스 게이트
16. 운영·관찰 가능성·비용 통제
17. 안티패턴과 최종 권고
18. 부록: 소스 레지스터, 코드 교차참조, 샘플 계약, 용어집

1 경영·아키텍처 요약

핵심 결론. foundry-lite 에 필요한 것은 별도의 챗봇 서비스를 추가하는 일이 아니라, 이미 구현된 데이터 ·Ontology·Object·Action·Workflow 페루프 위에 **AIP-lite 실행 계층**을 올리는 일이다. 현재 코드베이스는 불변 데이터 버전, 객체·링크·액션 모델, 권한 적용 조회, 의미 검색, 낙관적 동시성, 멍등성, 외부 writeback, audit, outbox, lineage, Temporal, AI evidence, human review 등 AIP 의 기반 중 가장 어려운 부분을 상당수 보유한다. 반면 Model Gateway, Model Catalog, Agent Definition, Context Compiler, Tool Broker, AI 실행 원장, AI 전용 보안 정책, Evals·Release Governance 는 사실상 신규 구축 영역이다.

1.1 최종 설계 명제

LLM 은 시스템의 결정권자나 데이터 접근 계층이 아니라 비신뢰 계획자(untrusted planner)다. 모든 데이터 조회는 기존 권한 적용 서비스 경계를 통과하고, 모든 운영 변경은 기존 ActionService 를 통해서만 일어나며, 고위험 변경은 정규화된 proposal 과 human review 를 거친다.

이 원칙을 지키면 AIP 가 기존 플랫폼을 우회하지 않고, Ontology·Action·Audit 의 보증을 그대로 상속한다. 반대로 LLM 이 repository, SQL, 임의 HTTP write 를 직접 수행하도록 두면 지금까지 구축한 트랜잭션·권한·lineage·writeback 보증이 무력화된다.

1.2 현재 준비도 요약

영역	준비도	판정
Data / Ontology 기반	85%	거의 그대로 재사용 가능
Object Query / Search	80%	AI tool wrapper 와 security partition 강화 필요
Action Runtime	85%	AIP write 경계로 직접 재사용 가능
Document / RAG 기반	65%	검색·버전·hash 는 있으나 Context Compiler 가 없음
Human Review	60%	review 는 있으나 승인 후 Action 실행 브리지가 없음
Durable Workflow	65%	Temporal 기반은 있으나 pause/signal 계약 확장 필요
Audit / Lineage	65%	범용 원장은 강하지만 AI 전용 event ledger 가 없음
AI 보안 모델	45%	tenant/RBAC/masking 은 있으나 egress·tool policy 부족
Model Gateway / Catalog	10%	신규 구축
Agent Runtime	10%	신규 구축
Evals / AI Release	15%	evidence 규약만 있고 평가·승격 체계는 없음

1.3 우선순위가 가장 높은 세 가지 위험

R1. 검색 projection 안의 민감정보. Object search projection 을 만드는 시스템 context 가 admin, finance 역할을 사용하므로, finance/PII property 가 searchable/indexed 로 선언된 경우 외부 Elasticsearch projection 에 저장될 가능성이 있다. 조회 후 masking 만으로는 provider 반출과 ranking side channel 을 완전히 차단하지 못한다. 이는 코드 조합에 기반한 강한 추론이다.

R2. Content index 보안 envelope 부족. content_units 는 security envelope 를 보유하지만 IndexedContentUnit 과 HybridContentQuery 는 tenant 외의 security token 을 담지 않는다. 동일 tenant 안의 classification·group·purpose 분리가 필요한 환경에서는 retrieval pre-filter 가 부족하다.

R3. 승인과 Action 실행의 단절. InsightReviewService 는 proposal 을 저장하고 approve/reject 하지만 승인된 proposal 을 ActionService.apply_action()으로 연결하는 실행 경로가 없다. AI 변경의 안전한 페루프를 완성하려면 exact proposal fingerprint 승인, 승인 시 재검증, Action 실행, 실행 결과 연결이 필요하다.

1.4 권장 도입 순서

검색 보안 강화

- Model Gateway / Model Alias
- AI Run & Event Ledger
- Context Compiler / Retrieval Orchestrator
- Read-only Tool Broker
- Citation Service
- Action Proposal
- Approval-to-Action Bridge
- Evals / Release Governance
- Logic Runtime
- Visual Builder

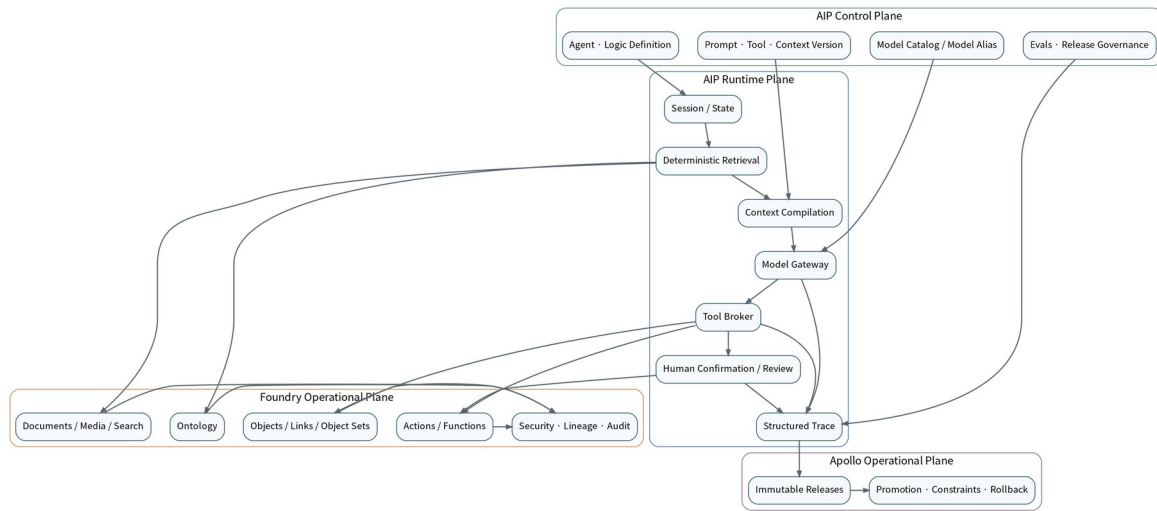


그림 1. Palantir AIP 를 공개 동작 기준으로 복원한 계층 구조

2 조사 범위와 판독 원칙

2.1 조사 기준일과 기준 브랜치

- 공개 문서 기준일: 2026-06-25
- 코드 기준: ludia8888/foundry-lite, default branch main
- 구현 경계의 우선 원본: docs/implementation-status.md
- 설계 의도의 우선 원본: foundry_lite_development_plan_ko_sprintified.md
- 외부 Foundry 분석의 기준 원본: deep-research-report.md
- 세부 코드 검토 대상: application services, ports, infrastructure composition root, SQLAlchemy schema, quality gates

2.2 근거 구분

표기	의미	사용 원칙
P-xx	Palantir 공식 공개 문서	제품 계약과 공개 동작 확인
C-xx	foundry-lite 코드 또는 내부 문서	현재 구현의 실제 경계 확인
아키텍처 추론	공개 동작과 코드에서 합리적으로 복원	비공개 구현으로 단정하지 않음
권고안	foundry-lite 에 도입할 목표 설계	구현 전 ADR·테스트로 확정

2.3 분석에서 의도적으로 하지 않은 것

- Palantir 내부 DB 제품이나 메시지 버스를 특정하지 않는다.
- 공개되지 않은 서비스명이나 마이크로서비스 수를 만들어내지 않는다.
- UI 유사성을 기능 패리티로 간주하지 않는다.
- LLM 성능을 플랫폼 아키텍처 완성도와 동일시하지 않는다.
- 계획 문서의 체크박스를 현재 구현 완료 증거로 사용하지 않는다.

3 Palantir AIP 작동원리의 복원

3.1 AIP·Foundry·Apollo의 역할 분리

Palantir 공식 아키텍처는 Foundry를 데이터 관리, 로직 작성, Ontology 개발, 분석, workflow 개발의 기반 플랫폼으로, AIP를 secure LLM connectivity, agents·automations 개발 도구, AI 애플리케이션, Evals를 제공하는 생성형 AI 플랫폼으로 설명한다. AIP와 Foundry는 통합 운영 시스템을 이루고, Storage·Compute·Networking·Security·Governance·Workspace 같은 횡단 기능은 Apollo에 의해 운영된다. [P-01]

실질적인 책임 분리는 다음과 같이 해석할 수 있다.

계층	주 책임	foundry-lite에 주는 시사점
Foundry Operational Plane	데이터, Ontology, Object, Action, Workflow, lineage	기존 코어를 진실 원본으로 유지
AIP Control Plane	모델·agent·prompt·tool·eval·release 정의	신규 immutable definition registry 필요
AIP Runtime Plane	context, model call, tools, state, trace	신규 runtime service와 AI ledger 필요
Apollo Plane	package, promotion, constraints, rollback	agent release channel과 환경별 승격 필요

Palantir의 2026년 AIP architecture overview는 secure model access, end-to-end observability, context engineering, Ontology, vector/compute/tool services, security/governance, agent lifecycle, operational automation, development environments, human+AI applications, package/release/deploy, enterprise automation의 12개 범주를 명시한다. [P-02]

3.2 Ontology가 AIP의 중심인 이유

Ontology는 단순한 의미 계층이 아니라 데이터, 로직, 액션, 보안을 통합하여 인간과 AI agent가 동일한 운영 세계에서 협업하도록 만드는 시스템으로 설명된다. [P-03] 이 설명은 AIP가 별도의 AI용 데이터베이스를 만드는 구조보다, 권한과 동작이 정의된 Ontology를 LLM의 context와 tool surface로 사용한다는 해석을 지지한다.

AIP에서 Ontology는 다음 네 역할을 동시에 수행한다.

1. **Context vocabulary:** 객체, 속성, 링크가 LLM이 이해할 운영 명사를 제공한다.
2. **Query boundary:** agent가 조회 가능한 object type과 property를 제한한다.
3. **Action boundary:** 운영 변경을 typed action과 submission criteria에 묶는다.
4. **Security boundary:** 인간과 agent가 같은 권한·marking·purpose 통제를 받는다.

3.3 Chatbot runtime의 핵심: prompt compilation

AIP Chatbot Studio는 instructions, tool descriptions, application-variable descriptions를 raw system prompt로 컴파일한다고 공개한다. LLM은 명시적으로 제공된 정보에만 접근할 수 있으며, context window에는 system prompt, 대화 이력, retrieval context, application state, tool definitions가 함께 들어간다. [P-04]

이를 실행 단계로 복원하면 다음과 같다.

```
Published Chatbot Definition
+ Instructions
+ Tool descriptions / schemas
+ Application-state descriptions
+ Deterministically retrieved context
+ Conversation history
+ User message
= Compiled model request
```

foundry-lite 에 적용할 때는 문자열 합치기로 구현해서는 안 된다. 각 구성요소의 버전, hash, 노출 정책, token budget 을 기록하는 Context Compiler 가 필요하다.

3.4 Retrieval context 는 매 메시지의 결정론적 단계

Palantir 는 retrieval context 가 매 새 사용자 메시지마다 결정론적으로 실행되고 결과가 LLM 으로 전달된다고 명시한다. 지원 context 는 Ontology, document, function-backed context 다. Ontology context 는 정적 object set 또는 semantic search 결과를 사용할 수 있고, LLM 에 전달할 property 를 선택할 수 있다. Document context 는 전체 문서 또는 관련 chunk 를 제공하며, function-backed context 는 사용자 정의 hybrid retrieval 을 허용한다. [P-05]

이 구조의 중요한 의미는 다음과 같다.

- retrieval 은 모델이 필요하면 호출하는 선택적 tool 과 구분된다.
- 권한을 적용한 context 가 모델 호출 전에 확정될 수 있다.
- 매 turn 의 context manifest 를 재현할 수 있다.
- tool call 없이도 Ontology 또는 문서 근거를 지속적으로 제공할 수 있다.

3.5 Application state 와 deterministic inputs

AIP Chatbot state 는 string 또는 object-set 변수로 구성할 수 있다. 변수 설명은 prompt 에 주입되며, 값의 visibility 를 별도로 설정할 수 있다. Ontology retrieval 또는 object query 의 deterministic input 으로 object set 을 사용할 수 있고, 같은 reasoning loop 의 초기 값에 pin 할 수도 있다. [P-06]

foundry-lite 에 필요한 직접적인 설계 원칙은 다음과 같다.

- state description 과 state value 를 분리한다.
- 값은 필요한 경우에만 LLM 에 노출한다.
- object 전체를 세션에 복제하지 않고 ID 또는 ObjectSet reference 를 보존한다.
- 같은 turn 에서 deterministic input 에 사용된 state snapshot 은 변경되지 않는다.
- state update 는 tool call 로 처리하고 audit 한다.

3.6 Tool 호출과 제어 흐름

AIP Chatbot tools 에는 Action, Object Query, Function, application-variable update, Command, clarification request 가 포함된다. Action 은 자동 또는 사용자 confirmation 후 실행할 수 있으며, Object Query 는 허용 object type 과 property 를 제한하고 filtering, aggregation, inspection, link traversal 을 지원한다. Prompted tool calling 과 native tool calling 이 모두 존재한다. [P-07]

AIP Logic 의 LLM block 도 prompt, tools, output 으로 구성되며, query objects, action, function, calculator 등의 tool 을 제공한다. Logic 은 LLM block 외에도 deterministic Apply Action, Execute Function, conditional, loop, variable block 을 연결한다. [P-08]

따라서 AIP 의 tool 구조는 다음 두 유형을 분리한다고 볼 수 있다.

유형	설명	foundry-lite 구현
Agent-selected tool	LLM 이 필요성과 argument 를 제안	ToolBroker 검증 후 기존 service 호출
Deterministic block	DAG 정의가 직접 action/function 을 호출	Logic Runtime 이 schema 검증 후 호출

3.7 Action 은 AI write 의 트랜잭션 경계

Palantir Action 은 객체·property·link 변경과 side effect 를 하나의 transaction 으로 묶는 운영 추상화다. Action 실행 가능 여부는 object/link/datasource visibility 와 submission criteria 에 의존한다. Submission criteria 는 사용자·그룹과 parameter 값을 결합할 수 있다. [P-09] [P-10]

이로부터 다음을 도출할 수 있다.

AIP 가 안전한 이유는 LLM 의 출력이 신뢰할 수 있어서가 아니라, 실제 변경이 Ontology Action 의 permission, parameter schema, criteria, transaction, side-effect 경계를 통과하기 때문이다.

3.8 Citations 와 context source identity

AIP Chatbot 은 context source key 를 citation 형식으로 응답에 포함하도록 가르치며, object citation 또는 custom document citation 을 렌더링한다. [P-11] foundry-lite 에서는 모델이 임의 URL 이나 문자열 citation 을 생성하도록 두기보다, Context Compiler 가 발급한 opaque context ID 를 모델이 인용하고 CitationService 가 authoritative source link 로 변환해야 한다.

3.9 Session logging 과 observability

Palantir 는 각 chatbot message execution 에 unique trace identifier 를 부여하고 structured event 로 export 한다. event 에는 session metadata, user request 와 retrieval context, compiled system prompt 와 tool definition, user message 등이 포함된다. AIP observability 는 metrics, execution history, distributed tracing, logging, log search 를 Workflow Lineage 에서 제공한다. [P-12] [P-13]

이는 foundry-lite 의 AI logging 이 단일 audit_event 한 행으로 충분하지 않음을 의미한다. 모델 호출, retrieval, tool call, human gate, final response 를 별도 event 로 저장하되 같은 trace 로 묶어야 한다.

3.10 Model Catalog 와 provider proxy

Palantir Model Catalog 는 사용자 enrollment 에서 사용 가능한 LLM 을 탐색·비교·시험하는 application 이며, provider-compatible proxy endpoint 는 native API request shape 을 유지하면서 rate limiting, data governance, usage tracking 을 제공한다. [P-14] [P-15]

foundry-lite 에서는 이 패턴을 다음과 같이 축소 적용해야 한다.

```
Agent Definition -> Model Alias -> Resolved Provider/Model Revision
-> Rate / Cost / Egress Policy
-> Provider-compatible Adapter
```

3.11 Evals 와 release confidence

AIP Evals 는 test case, evaluation function, model comparison, version comparison, repeated-run variance 를 지원한다. 목표는 비결정적 LLM-backed function 을 production 에 배치할 신뢰를 만드는 것이다. [P-16]

따라서 prompt 변경은 일반 configuration 수정이 아니라 release candidate 변경으로 취급해야 한다.

3.12 Apollo 의 시사점

Apollo 는 다양한 연결성·규제 환경에 소프트웨어를 안전하게 배포하고, 중앙 change management 와 autonomous deployment 를 제공한다. [P-17] foundry-lite 가 Apollo 자체를 복제할 필요는 없지만 다음 원칙은 AI asset 에 적용해야 한다.

- immutable version
- environment-specific constraint
- release channel
- promotion approval
- rollback pointer
- compatibility manifest

4 foundry-lite 현행 아키텍처 진단

4.1 제품 정체성과 페루프

foundry-lite 의 제품 문서는 시스템을 BI/ETL 도구가 아니라 Operational Object System 으로 정의한다. 현재 핵심 흐름은 file/snapshot 또는 connector 에서 데이터가 들어와 immutable dataset version 이 되고, transform, Ontology mapping, object indexing, query, Action, object edit, action log, materialization, downstream transform 으로 이어지는 페루프다. [C-01]

```

Ingest
→ Immutable Dataset
→ Transform
→ Ontology Mapping
→ Object / Link Index
→ Query / Search
→ Action Runtime
→ Object Edit / Writeback / Outbox
→ Materialization
→ Downstream Transform
    
```

이 페루프는 AIP 가 tool 을 통해 읽고 쓰는 운영 세계로 바로 활용할 수 있다.

4.2 Modular monolith 와 port/adaptor 경계

CoreDependencies 는 storage, compute, connector, search, embedding, media, stream, workflow, secret provider 를 port 로 주입한다. CoreServices 는 Dataset, Object, Media 등의 service group 을 만들고 collaborator 를 명시적으로 결합하며, FoundryLite root facade 가 bounded-context facade 를 노출한다. [C-02] [C-03] [C-04]

이 구조는 AIP 를 별도 마이크로서비스로 먼저 분리하기보다 같은 modular monolith 안에 AipServices bounded context 로 추가하기에 적합하다. AI 가 기존 service 와 transaction boundary 를 직접 재사용할 수 있고, provider SDK 만 infrastructure adaptor 에 격리할 수 있기 때문이다.

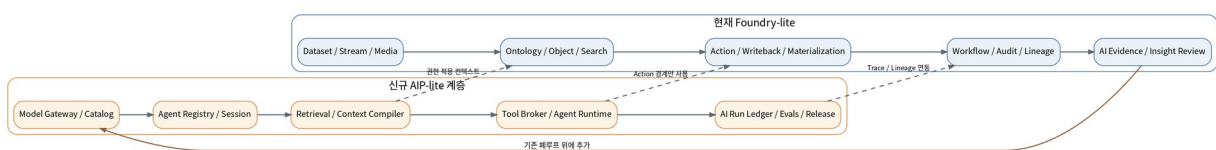


그림 2. 기존 Foundry-lite 페루프와 신규 AIP-lite 계층의 관계

4.3 Data·storage·compute 계층

현재 구현 상태 문서는 local SQLite/filesystem 을 기본으로 하면서 S3/MinIO, Iceberg, Spark, Kafka-compatible stream, Debezium CDC, Elasticsearch, Temporal 등의 adapter proof 가 존재한다고 명시한다. Dataset transaction 과 manifest commit, lineage, rollback·orphan cleanup, pinned input version, failure-abort invariant 가 강화되어 있다. [C-05]

AIP 관점에서 이 부분의 가치는 다음과 같다.

- context source 를 immutable dataset/media/object version 에 pin 할 수 있다.
- batch·stream·CDC 데이터를 같은 Ontology 로 투영할 수 있다.
- AI 추출 결과도 다른 transform output 과 같은 lineage 규약에 넣을 수 있다.
- retrieval index 를 source of truth 가 아닌 rebuildable projection 으로 다룰 수 있다.

4.4 Ontology · Object Query

ObjectQueryService 는 object:read permission 을 요구하고, active object type 을 조회하며, authoritative object record 에 policy masking 을 적용한다. include_explain 은 별도의 object:explain permission 이 있는 경우에만 base/edit layer 와 lineage 를 제공한다. Filter 와 orderBy 에서 masked property 를 참조하는 요청도 거부한다. [C-06]

AI tool 로 재사용할 핵심 API 는 다음과 같다.

```
get_object
query_objects
get_links
search_objects
search_objects_semantic
object_set create/list/resolve
```

중요한 원칙은 Agent Runtime 이 repository 를 호출하지 않고 이 application service 를 호출해야 한다는 것이다.

4.5 Object Search 와 semantic projection

Object Search 는 ontology property metadata 에서 indexed/searchable field 를 만들고, embedding model 이 있을 때 searchable property 의 텍스트를 결합해 model-version-pinned vector 를 생성한다. 검색 후에는 active object record 를 다시 읽고 policy masking 을 적용하며 projection version 과 object version 을 비교한다. [C-07] [C-08]

이는 RAG 의 authoritative re-read 패턴으로 매우 적합하다. 다만 projection 을 만들 때 사용하는 system context 와 property classification 의 결합은 보안 갭을 만든다. 해당 위험은 뒤에서 별도로 다룬다.

4.6 Media · Document · RAG 기반

Media subsystem 은 catalog, transaction, upload, processing, indexing, retrieval, visual search, binding, retention 으로 분리되어 있다. Processor output 은 source media item version, processor/model version, parameter hash 에 pin 되고, content unit 은 page/time/bbox/text hash/chunk spec/security envelope 를 보존한다. [C-09] [C-10]

DefaultContentRetrievalService 는 search index 결과를 그대로 반환하지 않고 DB content unit 을 재조회하여 tenant 와 text hash 를 확인한다. EmbeddingModelAdapter 는 embedding 을 immutable content unit 에서 파생되는 projection 으로 명시하고 model version 을 index generation 에 pin 한다. [C-11] [C-12]

AIP RAG 에서 재사용 가능한 요소:

- immutable source version
- content unit identity
- page/time/bbox citation coordinates
- text hash integrity
- embedding model version
- shadow generation promotion
- authoritative re-read

추가해야 할 요소:

- 최종 prompt 용 text payload
- relevance score 와 retrieval method
- classification/security partition
- token estimate
- retrieval trace
- citation rendering contract

4.7 Action Runtime

ActionService.apply_action()은 action 별 permission, write gate, active action definition, target type invariant, idempotency replay, expected object version, parameter validation, precondition, writeback, mutation unit of work, audit 를 결합한다. External writeback 은 outcome unknown 과 compensation required 를 구분한다. [C-13]

safe_expression.py 는 required parameter, unknown parameter rejection, primitive type check, precondition 을 수행한다. [C-14]

AIP 의 write tool 에서 그대로 상속할 수 있는 보증:

- request fingerprint
- idempotency key
- optimistic concurrency
- typed action parameter
- precondition
- no partial internal commit
- writeback reconciliation
- audit and outbox

4.8 Human Review

InsightReviewService 는 evidence object 와 evidence reference 가 있는 claim 만 생성하고, create/assign/approve/reject 를 idempotent key 로 관리한다. action_proposal 필드도 보유한다. [C-15]

그러나 service collaborator 가 없으며 approval 은 review row 의 상태만 바꾼다. 승인된 exact proposal 을 재검증하고 Action 으로 실행하는 application service 가 필요하다.

4.9 Workflow 와 Temporal

WorkflowOrchestrationService 는 external workflow 를 시작하기 전에 Foundry-owned run ledger 에 intent 를 기록한다. tenant/workflow/idempotency namespace 와 request fingerprint 를 사용하고, adapter start failure 를 durable state 로 남긴다. [C-16]

WorkflowAdapter 는 start 와 lookup 만 정의하며, Temporal adapter 는 stable workflow ID, duplicate rejection, start-and-wait, typed timeout/unavailable/business failure 를 제공한다. [C-17] [C-18]

장기 human approval 을 위해서는 signal, cancel, query, async start contract 가 추가로 필요하다.

4.10 Security 와 policy

현재 PolicyService 는 tenant context, 정적 role-based permission, ontology classification 기반 masking 을 제공한다. finance 와 admin 은 finance/PII 값을 읽을 수 있고, 다른 role 은 mask 된다. Operation detail 은 operator role 로 제한된다. [C-19]

장점:

- fail-closed default: 정의되지 않은 permission 은 admin-only
- ontology-driven sensitive property discovery
- query/filter 에서 masked property 차단
- preview, object, audit 등에 공통 masking

한계:

- agent 별 tool allowlist 없음
- purpose/marketing/clearance 모델 없음
- provider egress policy 없음
- parameter-dependent AI submission policy 없음
- action permission 일부가 static name 으로 하드코딩

4.11 Audit·Lineage·Operations

RuntimeService 는 lineage edge, audit event, outbox event, run relation 을 저장하고, run detail 에서 related evidence 와 downstream impact 를 결합한다. Runtime run type 에는 sync, transform, index, action, materialization, workflow 등이 있으나 AI run type 은 없다. [C-20] [C-21]

AI execution 을 단일 audit payload 에 넣기보다 별도 event ledger 를 만들고 기존 runtime relation 과 연결해야 한다.

4.12 AI evidence

ai_evidence.py 는 source dataset/object version, source span, extractor/model/prompt version, parameter hash, human review status, revision chain 을 모델링한다. Reprocessing 은 이전 evidence 를 덮어쓰지 않고 새 revision 을 만든다. [C-22]

이 코드는 AIP runtime 전체가 없더라도 AI result 의 provenance 를 어떻게 저장해야 하는지 이미 좋은 방향을 제시한다.

5 AIP 와 foundry-lite 의 기능·구조 교차대조

5.1 전체 매핑

Palantir AIP 기능	현재 foundry-lite 대응	상태	필요한 조치
Ontology context	ObjectQuery/ObjectLinks/ObjectSearch	강함	ContextProvider wrapper
Document context	Media processing/content index/retrieval	부분	security partition, context DTO
Function-backed context	application services/ports	부분	approved function registry
Application state	ObjectSet 은 존재, session state 없음	부족	versioned session/state store
Object Query tool	ObjectQueryService	강함	ToolSpec 와 Agent allowlist
Action tool	ActionService	강함	proposal · confirmation · review gate
Function tool	bounded services 존재	부족	allowlisted function tool adapter
Model Catalog	없음	신규	model/provider/alias registry
Provider proxy	없음	신규	LanguageModelAdapter/Gateway
Prompt compilation	없음	신규	Context Compiler
Session logging	generic audit/runtime	부분	AI event ledger
Citation	source ID/hash 존재	부분	CitationService
AIP Logic	Workflow/Temporal 기반	부분	typed DAG, block runtime
Evals	AI evidence unit tests	매우 부족	suite/case/run/result
Release channel	adapter profiles/CI gates	부분	agent/prompt/model manifest
Human review	InsightReview	부분	approval execution bridge

5.2 재사용해야 하는 기존 경계

결정 1. Object context 는 새 repository 가 아니라 ObjectQueryService, ObjectLinkService, ObjectSearchService 를 감싼다. 이 서비스들이 tenant, active index, property masking, stale projection 처리의 원본이다.

결정 2. Document context 는 ContentIndexAdapter 결과를 DB content unit 으로 다시 검증한 후에만 prompt 에 넣는다. index hit 는 candidate 이지 진실 원본이 아니다.

결정 3. 모든 write 는 ActionService 를 통해서만 수행한다. Agent Runtime 은 object repository, SQLAlchemy engine, connector write API 를 직접 주입받지 않는다.

결정 4. AI runtime 은 existing RuntimeService 를 대체하지 않고 AI-specific event ledger 를 추가하여 relation 으로 연결한다. 기존 Operations UI 와 downstream impact 분석을 유지한다.

6 핵심 갭과 우선 위험

6.1 Model Gateway 부재

현재 dependency graph 에는 embedding 과 vision model port 는 있으나 general language model port 가 없다. 일반 모델 호출에 필요한 기능은 다음과 같다.

- provider request/response normalization
- model alias resolution
- timeout/retry/failure taxonomy
- rate limit
- token/cost accounting
- provider request ID
- egress eligibility
- secret version reference
- streaming
- native tool calling capability
- structured-output capability

Provider SDK 를 FastAPI route 나 Agent Runtime 에서 직접 호출하면 adapter boundary 가 무너진다.

6.2 Agent Definition 과 immutable release 부재

현재 prompt, tool allowlist, context source, budget, output schema 를 하나의 published agent version 으로 고정하는 저장소가 없다. 이 상태에서 prompt 를 environment variable 이나 mutable YAML 로 두면 재현성과 rollback 이 불가능하다.

6.3 Context Compiler 부재

현재 object/document search 는 존재하지만 다음을 하나의 deterministic model request 로 조합하는 계층이 없다.

- agent instruction
- application-state description/value
- tool schemas
- retrieved object/document/function context
- citation IDs
- token budget
- policy snapshot
- conversation summary/history

Context Compiler 는 보안 경계이자 provenance 경계다.

6.4 Tool Broker 부재

기존 service 가 강하더라도 LLM 의 tool call 을 바로 service argument 로 넘겨서는 안 된다. 다음 검증이 별도로 필요하다.

- agent 가 해당 tool 을 보유하는지
- tool version 이 published 인지
- JSON schema validation
- user permission
- object/property allowlist
- masked property rejection
- risk tier
- confirmation/human review
- result size 와 timeout
- idempotency
- per-run budget

6.5 AI 실행 원장 부재

Generic audit 는 “누가 무엇을 했는가”에는 적합하지만, 한 AI turn 의 retrieval/model/tool loop 를 재구성하기에는 부족하다. 최소한 AI run, event, model call, context item, tool call, citation, usage ledger 가 필요하다.

6.6 Human approval 실행 브리지 부재

승인은 exact proposal fingerprint 에 대해 이루어져야 한다. 승인 시에도 현재 object version, action enabled 상태, reviewer permission, policy version, proposal expiry 를 다시 검사해야 한다.

6.7 Search security 의 구조적 위험

6.7.1 Object Search projection

현재 search projection 용 context 는 admin, finance role 을 가진다. PolicyService 는 이 role 에 민감 property 를 masking 하지 않는다. Search mapping 은 현재 context 에서 masked 가 아닌 indexed/searchable property 를 projection 에 넣는다. 따라서 active Ontology 에서 finance/PII property 가 searchable/indexed 이면 shared external index 에 저장될 수 있다. [C-07] [C-19]

권고:

1. sensitive property 는 shared index 에서 제외하거나,
2. clearance/security partition 별 index 를 만들거나,
3. security token pre-filter 를 adapter query 에 강제하고,
4. provider context 반출 시 별도의 egress redaction 을 수행한다.

6.7.2 Content Search projection

content_units 는 security envelope 를 저장하지만 IndexedContentUnit 과 query 에는 tenant 외의 security token 이 없다. Retrieval service 의 authoritative check 도 tenant 와 hash 중심이다. [C-10] [C-11]

권고 contract:

```
security_partition_ids
classification
allowed_principal_tokens
purpose_tags
policy_version
security_envelope_hash
```

Candidate ranking 전에 pre-filter 해야 하며, unauthorized candidate 의 존재가 score 나 top-k 경쟁에 영향을 주지 않아야 한다.

7 목표 AIP-lite 참조 아키텍처

7.1 설계 원칙

1. **Ontology-first**: context 와 tool 은 Ontology vocabulary 와 service 를 사용한다.
2. **LLM-untrusted**: 모델 출력은 proposal 이며, validation 없이 실행되지 않는다.
3. **Authoritative re-read**: index result 를 직접 prompt 나 action 에 사용하지 않는다.
4. **Identity propagation**: tool 은 최종 사용자 identity 와 tenant 를 유지한다.
5. **Immutable definitions**: model alias, prompt, agent, tool manifest, eval result 를 version 으로 고정한다.
6. **Evidence by default**: 모든 claim 과 action proposal 은 source reference 를 가진다.
7. **Bounded autonomy**: model/tool/loop/time/token/cost budget 을 강제한다.
8. **Separation of truth and projection**: vector/search cache 는 재구축 가능 projection 이다.
9. **Human gate for risk**: irreversible · external · financial writes 는 review 를 요구한다.
10. **Release before automation**: eval 과 promotion 없이 production autonomous agent 를 허용하지 않는다.

7.2 논리 구성

```

Client / UI
-> AIP API
  -> Agent Runtime
    -> Definition Resolver
    -> Session / State
    -> Retrieval Orchestrator
    -> Context Compiler
    -> Model Gateway
    -> Tool Broker
    -> Approval / Action Bridge
    -> Citation Service
    -> AI Run Ledger

기존 Foundry-lite
<- Object / Link / Search services
<- Media / Content services
<- ActionService
<- InsightReview
<- Workflow / Temporal
<- Runtime / Audit / Lineage
  
```

7.3 Control Plane

Control Plane 은 실행 코드가 아니라 “어떤 agent 를 어떤 모델 · prompt · tool · context · budget 으로 실행할 것인가”를 정의한다.

구성요소:

- model provider registry
- model entity 와 capability
- model alias 와 environment resolution
- prompt version
- agent definition/version
- context source configuration
- tool manifest
- output schema
- risk policy
- eval suite 와 release manifest

7.4 Runtime Plane

Runtime Plane 은 한 turn 또는 Logic run 을 실행한다.

- session and state
- AI run intent 선기록
- definition resolve
- deterministic retrieval
- authoritative re-read/masking
- context compilation
- model call
- tool authorization/execution
- bounded loop
- human gate
- citation resolution
- event trace commit

7.5 Operations Plane

- metrics, trace, logs
- usage/cost
- retry policy
- dead-letter/failed execution
- model/provider health
- prompt/context diff
- action-review conversion
- release rollback

8 핵심 컴포넌트 상세 설계

8.1 Model Gateway

8.1.1 책임

- model alias 를 실제 provider/model/revision 으로 resolve
- request normalization
- native/prompted tool calling capability negotiation
- structured output capability 확인
- provider secret resolve
- egress policy 확인
- timeout/rate limit/retry
- usage 와 latency 기록
- provider error 를 typed failure 로 변환

8.1.2 권장 port

```
class LanguageModelAdapter(Protocol):
    @property
    def profile_name(self) -> str: ...

    def complete(self, request: ModelRequest) -> ModelResponse: ...
    def stream(self, request: ModelRequest) -> Iterable[ModelEvent]: ...
    def failure_contract(self) -> AdapterFailureContract: ...
```

8.1.3 ModelRequest 필수 필드

```
model_alias
messages
tools
response_schema
temperature
max_output_tokens
request_id
ai_run_id
data_classification
region_requirement
timeout_seconds
```

8.1.4 ModelResponse 필수 필드

```
provider
resolved_model_id
resolved_model_revision
content
normalized_tool_calls
finish_reason
input_tokens
output_tokens
provider_request_id
latency_ms
```

8.1.5 Retry 원칙

- transport timeout · 429 · temporary unavailable 만 제한적으로 retry
- 같은 request fingerprint 와 idempotency context 를 유지

- tool-producing response 를 받은 뒤 model retry 금지 또는 별도 attempt chain 으로 기록
- write-producing agent 에서 다른 model alias 로 silent fallback 금지

8.2 Model Catalog 와 Alias

Agent 는 provider-native model ID 가 아니라 alias 를 참조한다.

```
order-copilot-fast -> provider-a/model-small@revision-12
order-copilot-smart -> provider-b/model-large@revision-7
```

Alias record 에는 다음이 필요하다.

- lifecycle: draft/enabled/deprecated/disabled
- environment mapping
- capabilities: streaming, native tools, parallel tools, JSON schema, vision
- context/output token limits
- provider region
- allowed classifications
- retention/training assurance metadata
- rate/cost budget
- eval evidence

8.3 Agent Definition Registry

```
apiName: order_operations_copilot
version: 7
status: published
modelAlias: order-copilot-smart
systemPromptVersion: order-copilot-system@12
context:
  objectTypes: [PurchaseOrder, Supplier, Shipment]
  documentSources: [supplier_contracts]
  maxContextTokens: 18000
tools:
  - ontology.get_object@1
  - ontology.query_objects@1
  - ontology.get_links@1
  - content.search@1
  - action.propose.ApproveOrder@1
budgets:
  maxModelCalls: 5
  maxToolCalls: 8
  maxInputTokens: 50000
  maxOutputTokens: 6000
  maxExecutionSeconds: 90
```

Published version 이 반드시 고정할 항목:

- prompt version
- model alias version
- tool manifest version
- Ontology compatibility
- context policy
- output schema
- risk tier
- budget
- release channel

8.4 Session 과 State

초기에는 다음 타입으로 제한하는 것이 좋다.

```
scalar
object_set_ref
```

State row 예시:

```
{
  "selectedSupplierId": {"type": "scalar", "value": "supplier-217"},
  "ordersUnderReview": {
    "type": "object_set_ref",
    "objectType": "PurchaseOrder",
    "objectIds": ["po-1", "po-2"]
  }
}
```

원칙:

- session state 는 versioned snapshot
- same-turn deterministic input 은 initial snapshot 에 pin
- object value 는 매 turn current permission 으로 재조회
- state description 과 value visibility 를 분리
- state update 도 tool event 로 기록
- 대화 요약은 derived artifact 로 source message IDs 와 model/prompt version 을 보존

8.5 Retrieval Orchestrator

8.5.1 Pipeline

```
User query
-> deterministic normalization
-> lexical candidates
-> dense candidates
-> rank fusion
-> optional reranking
-> authoritative DB re-read
-> security validation
-> version/hash validation
-> dedup/diversity
-> token-budget packing
-> RetrievedContextItem[]
```

초기 ranking 은 BM25 + dense cosine + Reciprocal Rank Fusion 으로 충분하다. HyDE, query augmentation, LLM reranker 는 baseline eval 후 도입한다. Palantir 도 OAG 문서에서 ranked keyword, query augmentation, hybrid search, HyDE 를 개선 기법으로 제시한다. [P-18]

8.5.2 최종 context DTO

```
@dataclass(frozen=True)
class RetrievedContextItem:
    context_id: str
    kind: Literal["object", "document", "function"]
    text: str
    source_ref: SourceRef
    source_version: str
    content_hash: str
    relevance_score: float
    retrieval_method: str
    security_partition: str
    token_estimate: int
```

8.5.3 Retrieval trace

```
{
  "queryHash": "sha256:...",
  "filters": {},
  "lexicalCandidateCount": 40,
  "denseCandidateCount": 40,
  "selectedContextIds": ["ctx_1", "ctx_7"],
  "omittedBySecurityCount": 3,
  "omittedByBudgetCount": 8,
  "indexGeneration": "content-g17",
  "embeddingModelVersion": "bge-small@3"
}
```

운영 화면에는 unauthorized candidate 의 ID 나 text 를 노출하지 않는다.

8.6 Context Compiler

8.6.1 고정 순서

1. Platform safety policy
2. Published agent instruction
3. State schema and visible values
4. Tool definitions
5. Authoritative retrieved context
6. Citation mapping
7. Output schema
8. User message

8.6.2 생성 hash

```
compiled_prompt_hash
context_manifest_hash
tool_manifest_hash
state_snapshot_hash
policy_snapshot_hash
```

8.6.3 Prompt injection 격리

검색 문서는 instruction 이 아니라 untrusted data 다.

```
<context-item
  id="ctx_17"
  kind="document"
  source-version="media_version_42"
  page="11"
  trust="untrusted-data">
  ...document text...
</context-item>
```

Compiler 는 context item 안의 “이전 지시를 무시하라” 같은 문자열이 system instruction 으로 승격되지 않도록 명확한 delimiter 와 policy 를 사용해야 한다.

8.7 Citation Service

모델에는 source URL 대신 opaque context ID 를 제공한다.

```
Model output: ... [ctx_17]
CitationService:
ctx_17 -> media item version 42 / page 11 / content unit 3 / hash abc
```

검증:

- context ID 가 이번 run manifest 에 존재하는가
- 사용자에게 source 를 볼 권한이 있는가
- source version 과 hash 가 여전히 일치하는가
- citation span 이 claim 과 무관하지 않은가

최종 API 는 citation display payload 와 signed navigation reference 를 반환한다.

8.8 Tool Registry 와 Tool Broker

```
@dataclass(frozen=True)
class ToolSpec:
    tool_id: str
    version: str
    input_schema: JsonSchema
    output_schema: JsonSchema
    effect: Literal["READ", "PROPOSE_WRITE", "WRITE"]
    required_permission: str
    confirmation_policy: Literal["NONE", "USER", "HUMAN_REVIEW"]
    object_type_allowlist: tuple[str, ...]
    property_allowlist: tuple[str, ...]
    timeout_seconds: int
    max_result_items: int
```

ToolBroker 검사 순서:

1. Agent tool allowlist
2. published tool version
3. input JSON schema
4. user permission
5. object/property scope
6. masked property rejection
7. model egress compatibility
8. budget/timeout/result limit
9. confirmation/review requirement
10. idempotency and request fingerprint
11. output masking before returning to model

초기 tool:

```
ontology.get_object
ontology.query_objects
ontology.get_links
ontology.search_objects
content.search
state.update
action.propose
```

초기 금지 tool:

```
generic_sql
arbitrary_http_request
shell_execute
python_eval
generic_repository_write
```

8.9 Agent Runtime

Agent Runtime 은 다음 상태를 가진 bounded loop 로 구현한다.

```
RECEIVED
-> RESOLVING_DEFINITION
-> RETRIEVING_CONTEXT
-> MODEL_RUNNING
-> TOOL_PENDING
-> TOOL_RUNNING
-> WAITING_CONFIRMATION
-> WAITING_HUMAN_REVIEW
-> MODEL_RUNNING
-> SUCCEEDED
```

Terminal:

```
FAILED / CANCELLED / BUDGET_EXCEEDED / POLICY_DENIED
```

Budget:

- max model calls
- max tool calls
- max loop iterations
- max input/output tokens
- max wall-clock seconds
- max estimated cost
- max context items
- max tool output bytes

8.10 Action Proposal 과 Approval Execution

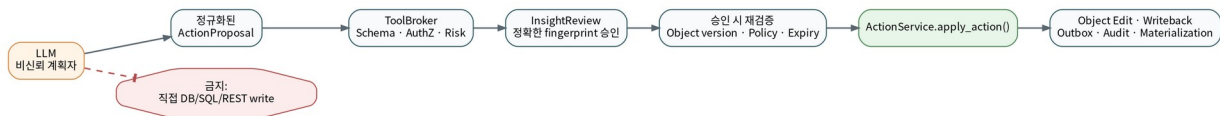


그림 3. AI 가 운영 변경을 수행하는 안전한 경로

정규화된 proposal:

```
@dataclass(frozen=True)
class ActionProposal:
    proposal_id: str
    action_type: str
    target_object_type: str
    target_object_id: str
    expected_object_version: int
    parameters: Mapping[str, object]
    evidence_refs: tuple[EvidenceRef, ...]
    originating_ai_run_id: str
    proposal_fingerprint: str
    policy_version: str
    expires_at: str
```

Fingerprint 입력:

- action type
- target type/id
- expected object version
- canonical params

- evidence references
- agent version
- policy version

승인 시 재검증:

- proposal fingerprint 불변
- proposal expiry
- reviewer permission
- action enabled
- current object version
- source evidence access
- restore/write traffic gate
- current policy compatibility

승인 후 실행은 ApprovalExecutionService 가 수행한다. InsightReviewService 가 ActionService 를 직접 호출하기보다, 승인 이벤트와 execution ledger 를 분리하는 편이 concurrency 와 retry 를 다루기 쉽다.

8.11 Logic Runtime

초기 block 종류:

```
Input
RetrieveObject
QueryObjects
TraverseLinks
RetrieveContent
CallLLM
CallFunction
Condition
Map
Reduce
UpdateState
CreateActionProposal
ApplyAction
HumanApproval
Output
```

Publish-time validation:

- graph cycle 과 branch consistency
- input/output schema compatibility
- referenced object/action/function 존재
- Ontology compatibility
- model capability
- tool permission
- loop maximum
- output schema
- side-effect branch human gate
- context and cost budget

Temporal 사용 범위:

- 장기 human approval
- scheduled/batch Logic
- 대량 extraction/eval
- external side effect retry/compensation
- crash-safe pause/resume

Interactive chat turn 은 짧은 in-process bounded loop 가 더 적합하며, 긴 workflow 만 Temporal 로 넘긴다.

8.12 Evals 와 Release Governance

평가 축:

축	핵심 지표
Retrieval	Recall@K, Precision@K, MRR, security correctness
Answer	claim correctness, faithfulness, unsupported claim rate
Citation	resolvability, source-version match, coverage
Tool	selection, schema validity, unauthorized rejection
Action	target/params/version, review enforcement, idempotency
Security	tenant/property leakage, injection, egress, secret leakage
Operations	latency, tokens, cost, provider/tool failure

Agent release manifest 예시:

```
agentVersion: order-copilot@17
promptVersion: order-system@23
toolManifestVersion: order-tools@8
modelAliasVersion: order-smart@11
ontologyCompatibility:
  minimumVersion: 34
evalRunId: eval-run-991
releaseChannel: canary
```

Release channel:

```
draft -> dev -> canary -> stable -> sunset
```

Write-producing agent 는 deterministic security/action gate 와 repeated-run variance 를 통과해야 stable 로 승격된다.

9 보안·권한·데이터 반출 통제

9.1 최종 권한은 교집합

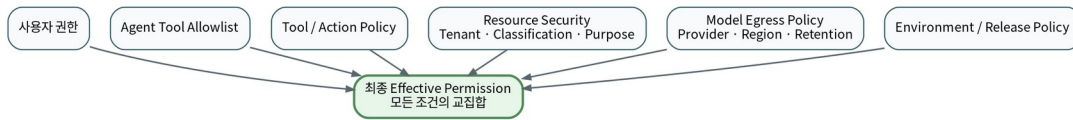


그림 4. AIP tool 실행에 필요한 권한 교집합

Effective Permission
 = User Permission
 $\cap$ Agent Definition Allowlist
 $\cap$ Tool / Action Policy
 $\cap$ Resource Security Policy
 $\cap$ Model Egress Policy
 $\cap$ Environment / Release Policy

사용자에게 Action 권한이 있어도 Agent definition 에 해당 Action 이 없으면 실행할 수 없어야 한다. 반대로 Agent 가 tool 을 보유해도 사용자에게 object/action permission 이 없으면 거부한다.

9.2 Identity propagation

- 모델 provider credential 은 시스템 credential 이지만 tool execution identity 는 최종 사용자다.
- service account 는 background index/eval worker 처럼 명시된 시스템 작업에만 사용한다.
- user-selected object set 도 매 turn 현재 사용자 권한으로 resolve 한다.
- approval reviewer 와 proposal creator 를 분리할 수 있어야 한다.

9.3 Model egress policy

분류별로 provider 와 region 을 허용한다.

분류	예시 정책
public	승인된 외부 provider 사용 가능
internal	계약된 zero-retention provider 만 가능
confidential	특정 region/provider 또는 self-hosted 만 가능
restricted/PII	raw value 반출 금지 또는 tokenization 후 허용

Egress decision 은 다음과 함께 기록한다.

provider
 model revision
 region
 classification set
 redaction/tokenization applied
 policy version
 decision reason

9.4 Retrieval security

- candidate retrieval 전 principal token pre-filter
- authoritative re-read 후 post-validation
- prompt 용 property allowlist
- classification 별 context cap

- unauthorized candidate existence 비노출
- index generation 별 security policy version pin
- model provider 에 보내기 직전 final egress redaction

9.5 Prompt injection 과 tool injection

방어층:

1. retrieved text 를 untrusted data 로 격리
2. system instruction 과 context delimiter 분리
3. 문서 안의 tool-call 형식을 실행하지 않음
4. native tool call 도 ToolBroker schema 를 통과
5. URL · command · SQL 문자열을 generic executor 로 전달하지 않음
6. tool result 를 다시 model 에 줄 때 크기 · field allowlist · masking 적용
7. repeated suspicious tool call 과 budget exhaustion 감지

9.6 Secret 보호

기존 SecretProvider 를 재사용한다. AI trace 에는 secret value 를 절대 저장하지 않고 다음만 남긴다.

```
secret_name
secret_version
provider_profile
resolution_timestamp
```

Provider error body 에 secret 이나 raw Authorization header 가 들어오지 않도록 adapter 에서 sanitize 한다.

9.7 Logging 과 privacy

Raw prompt 와 tool result 는 일반 audit JSON 에 무기한 저장하지 않는다.

권장 저장 정책:

- 일반 운영 DB: hash, redacted preview, counts, IDs
- encrypted prompt artifact: 별도 access permission
- retention: 짧고 명시적
- legal hold 연동
- erasure request 와 source lineage 연동
- exported AI log dataset 에 security marking 적용

Hidden chain-of-thought 는 저장 대상이 아니다. 저장할 것은 input, compiled context manifest, tool call/result, final answer, policy decision, 짧은 execution summary 다.

10 데이터 모델과 상태 머신

10.1 Control Plane 테이블

10.1.1 ai_model_providers

```
id, tenant_scope, provider_type, profile_name, region,
secret_ref, retention_policy, training_policy, status, created_at
```

10.1.2 ai_models

```
id, provider_id, provider_model_id, revision, lifecycle,
capabilities_json, context_limit, output_limit, pricing_json,
allowed_classifications, created_at
```

10.1.3 ai_model_aliases

```
id, alias, environment, model_id, version, status,
eval_run_id, effective_at, retired_at
```

10.1.4 ai_prompt_versions

```
id, api_name, version, template, template_hash,
input_schema, created_by, status, created_at
```

10.1.5 ai_agent_versions

```
id, agent_api_name, version, status, model_alias_version,
prompt_version_id, tool_manifest_hash, context_policy,
output_schema, budget, risk_policy, ontology_compatibility,
release_channel, created_at
```

10.1.6 ai_eval_*

```
ai_eval_suites
ai_eval_cases
ai_eval_runs
ai_eval_results
```

10.2 Runtime Plane 테이블

10.2.1 ai_sessions

```
id, tenant_id, agent_version_id, actor_user_id,
status, created_at, last_activity_at
```

10.2.2 ai_session_state_versions

```
id, tenant_id, session_id, version, state_json,
state_hash, created_by_run_id, created_at
```

10.2.3 ai_messages

```
id, tenant_id, session_id, role, client_message_id,
content_ref, content_hash, created_at
UNIQUE(tenant_id, session_id, client_message_id)
```

10.2.4 ai_execution_runs

```
id, tenant_id, session_id, agent_version_id, actor_user_id,
request_id, trace_id, status, ontology_version_id,
model_alias_version, resolved_model_id, resolved_model_revision,
prompt_version_id, compiled_prompt_hash, tool_manifest_hash,
context_manifest_hash, state_snapshot_hash, policy_snapshot_hash,
budget_json, usage_json, error_json, started_at, completed_at
```

10.2.5 ai_execution_events

```
id, tenant_id, ai_run_id, sequence, event_type,
payload_ref, payload_hash, redacted_preview, created_at
UNIQUE(ai_run_id, sequence)
```

10.2.6 ai_model_calls

```
id, tenant_id, ai_run_id, attempt, provider, model_revision,
request_hash, response_hash, provider_request_id,
input_tokens, output_tokens, latency_ms, status, error_json
```

10.2.7 ai_context_items

```
id, tenant_id, ai_run_id, context_id, kind,
source_resource_type, source_resource_id, source_version,
content_hash, retrieval_method, relevance_score,
security_partition, token_estimate, selected, omission_reason
```

10.2.8 ai_tool_calls

```
id, tenant_id, ai_run_id, sequence, tool_id, tool_version,
arguments_hash, effect, authorization_decision,
confirmation_policy, status, result_hash, linked_action_run_id,
started_at, completed_at, error_json
```

10.2.9 ai_citations

```
id, tenant_id, ai_run_id, message_id, context_item_id,
claim_span, citation_order, rendered_ref
```

10.2.10 ai_usage_ledger

```
id, tenant_id, ai_run_id, provider, model_revision,
input_tokens, output_tokens, estimated_cost, currency,
recorded_at
```

10.3 Insight review 확장

기존 insight_reviews 에 다음을 추가한다.

```
proposal_type
proposal_fingerprint
originating_ai_run_id
originating_tool_call_id
expires_at
execution_status
approved_action_run_id
approval_policy_version
```

10.4 AI event taxonomy

```

session.created
message.received
agent.resolved
state.snapshot
context.retrieval.started
context.retrieval.completed
prompt.compiled
model.call.started
model.call.completed
model.call.failed
tool.call.proposed
tool.call.authorized
tool.call.completed
tool.call.failed
action.proposal.created
human.review.requested
human.review.approved
action.executed
response.completed
execution.failed

```

10.5 Existing runtime 과 연결

기존 runtime_run_relations 의 run type literal 에 ai 또는 ai_execution 을 추가하거나, 별도 generalized relation table 을 도입한다.

관계 예시:

```

AI run --used--> object version
AI run --used--> content unit
AI run --called--> model call
AI run --proposed--> insight review
Insight review --approved_as--> action run
Action run --produced--> object edit
Action run --emitted--> outbox event

```

11 API·포트·패키지 구조

11.1 추가 ports

```
libs/foundry_lite/application/ports/
language_model.py
model_registry_repository.py
ai_definition_repository.py
ai_run_repository.py
context_provider.py
tool_executor.py
usage_meter.py
```

11.2 추가 services

```
libs/foundry_lite/application/services/aip/
model_gateway.py
definition_service.py
session_service.py
state_service.py
context_compiler.py
retrieval_orchestrator.py
tool_registry.py
tool_broker.py
agent_runtime.py
logic_runtime.py
citation_service.py
approval_execution.py
eval_service.py
```

11.3 추가 infrastructure

```
libs/foundry_lite/infrastructure/adapters/
provider_compatible_language_model.py
fake_language_model.py

libs/foundry_lite/infrastructure/repositories/
sqlalchemy_ai_definition_repository.py
sqlalchemy_ai_run_repository.py
sqlalchemy_model_registry_repository.py
```

11.4 Service composition

기존 MediaServices, ObjectServices 패턴과 동일하게 AipServices group 을 추가한다.

```
@dataclass(frozen=True)
class AipServices:
    model_gateway: ModelGatewayService
    definitions: AgentDefinitionService
    sessions: SessionService
    runtime: AgentRuntimeService
    logic: LogicRuntimeService
    evals: EvalService
```


FoundryLite facade:

```
foundry.aip.agents
foundry.aip.sessions
foundry.aip.runs
foundry.aip.evals
```

11.5 권장 API

11.5.1 Control Plane

```
POST /api/aip/models/aliases
GET /api/aip/models
POST /api/aip/prompts/versions
POST /api/aip/agents/versions
POST /api/aip/agents/{apiName}/publish
POST /api/aip/agents/{apiName}/promote
```

11.5.2 Runtime Plane

```
POST /api/aip/sessions
POST /api/aip/sessions/{sessionId}/messages
GET /api/aip/runs/{runId}
GET /api/aip/runs/{runId}/events
POST /api/aip/runs/{runId}/cancel
POST /api/aip/runs/{runId}/confirmations/{confirmationId}
```

11.5.3 Review / Action

```
POST /api/aip/proposals/{proposalId}/submit-review
POST /api/insight-reviews/{reviewId}/approve-and-execute
GET /api/aip/proposals/{proposalId}
```

11.5.4 Evals

```
POST /api/aip/evals/suites
POST /api/aip/evals/suites/{suiteId}/runs
GET /api/aip/evals/runs/{runId}
POST /api/aip/evals/runs/{runId}/compare
```

11.6 Error taxonomy

기존 AdapterFailureContract 패턴을 확장한다.

```
model: timeout | rate_limited | unavailable | content_rejected | schema_invalid | egress_denied
context: security_denied | budget_exceeded
tool: not_allowed | schema_invalid | policy_denied | confirmation_required
agent: budget_exceeded
proposal: expired | fingerprint_mismatch | approval_object_version_conflict
```

Error payload 에는 request_id, ai_run_id, retryability, operator message, safe details 를 포함한다.

12 종단간 실행 시퀀스

12.1 일반 Chat turn

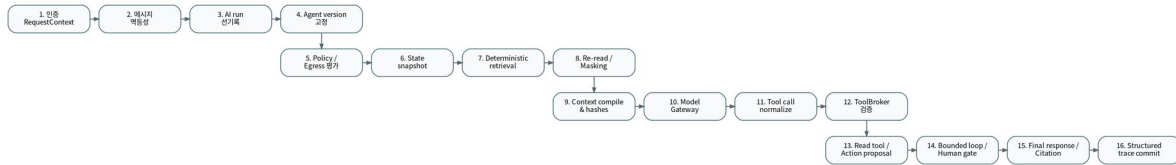


그림 5. 한 AI turn 의 권장 실행 순서

12.1.1 단계별 불변조건

1. **인증**: existing AuthProvider 로 RequestContext 를 생성한다.
2. **역등성**: (session_id, client_message_id) unique key 로 중복 turn 을 막는다.
3. **run 선거권**: provider 호출 전에 intent 를 durable 하게 기록한다.
4. **definition 고정**: draft 가 아닌 published agent version 만 실행한다.
5. **policy**: agent invoke, model use, context read, tool permission 을 평가한다.
6. **state snapshot**: same-turn deterministic input 을 고정한다.
7. **retrieval**: object/document/function context 를 결정론적으로 실행한다.
8. **authoritative validation**: DB source 와 hash, permission 을 다시 확인한다.
9. **compile**: prompt/context/tool/state/policy hash 를 생성한다.
10. **model call**: Model Gateway 만 통과한다.
11. **normalize**: provider 별 response 를 내부 message/tool-call 계약으로 변환한다.
12. **broker**: tool allowlist, schema, permission, risk, budget 을 검사한다.
13. **execute/propose**: read tool 은 실행, write 는 proposal 을 생성한다.
14. **bounded loop/human gate**: 최대 call · token · time 을 강제한다.
15. **citation**: opaque context IDs 를 source link 로 변환한다.
16. **trace**: 모든 event 와 usage 를 commit 한다.

12.2 Action approval flow

```

AI run
-> ActionProposal 생성
-> InsightReview pending
-> reviewer 가 exact fingerprint 승인
-> ApprovalExecutionService claim
-> current object/version/policy 재검증
-> ActionService.apply_action
-> action run / object edit / outbox / writeback
-> review execution_status=executed
-> AI run relation 연결
  
```

Concurrency:

- 하나의 review terminal decision 만 승리
- 하나의 proposal execution claim 만 승리
- Action idempotency key 는 proposal fingerprint 기반
- approval 후 object version 이 달라지면 실행하지 않고 conflict
- retry 는 동일 idempotency key 사용

12.3 Long-running Logic

```

API start
-> workflow intent ledger
-> Temporal async start
-> deterministic blocks
  
```

```
-> LLM block  
-> proposal  
-> WAITING_HUMAN_REVIEW  
-> reviewer decision signal  
-> version/policy recheck  
-> Action block  
-> terminal output
```

필요한 WorkflowAdapter 확장:

```
start_workflow_async  
signal_workflow  
cancel_workflow  
query_workflow  
workflow_run
```

13 첫 번째 제품 수직 슬라이스

13.1 Order Operations Copilot

13.1.1 사용자 질문

“PO-1042 가 지연된 이유와 재무 영향, 관련 계약 조항을 설명하고 승인 가능한 상태면 승인을 만들어줘.”

13.1.2 사용 context

- PurchaseOrder object
- linked Supplier
- linked Shipment
- relevant supplier contract content units
- object property lineage
- current action definition and preconditions

13.1.3 사용 tools

```
ontology.get_object(PurchaseOrder)
ontology.get_links(Supplier, Shipment)
content.search(supplier_contracts)
action.propose(ApproveOrder)
```

13.1.4 실행 결과

1. object versions 와 source dataset versions 를 pin 한다.
2. 계약 문서 page/content-unit/hash citation 을 확보한다.
3. 지연 원인과 재무 영향을 evidence-backed claim 으로 생성한다.
4. ApproveOrder proposal 에 expectedObjectVersion 을 포함한다.
5. InsightReview 를 생성한다.
6. reviewer 승인 후 current version 을 재검사한다.
7. existing ActionService 가 mutation/writeback/audit/outbox 를 수행한다.
8. materialized action log 로 downstream pipeline 에 환류한다.

13.1.5 Evidence graph

```
AI execution
├─ used_object_version -> PurchaseOrder@v17
├─ used_object_version -> Supplier@v4
├─ used_content_unit -> Contract/page-11/hash-...
├─ model_call -> model@revision
├─ proposed_action -> proposal-92
├─ requested_review -> insight-review-44
├─ approved_by -> reviewer-user
├─ executed_as -> action-run-311
├─ produced_edit -> object-edit-918
└─ produced_event -> outbox-221
```

13.2 수직 슬라이스 Acceptance Criteria

13.2.1 기능

- object · link · document 를 사용해 답변
- claim 마다 resolvable citation
- ApproveOrder proposal 생성
- review 승인 후 Action 실행
- materialization 에 반영

13.2.2 보안

- unauthorized object/property/document 가 provider 로 전달되지 않음
- reviewer 가 source 를 볼 수 없는 경우 승인 불가
- Action 권한과 agent tool allowlist 를 모두 만족
- external writeback 은 기존 reconciliation 보증 유지

13.2.3 재현성

- model/prompt/agent/tool/context/state/policy version pin
- 동일 turn 의 event order 재구성 가능
- citation source hash 검증 가능
- proposal fingerprint 와 Action request fingerprint 연결

13.2.4 운영

- token, latency, cost 기록
- failed model/tool/review/action 단계가 Operations 에서 구분됨
- retryable 여부가 명시됨

14 구현 로드맵과 PR 분해

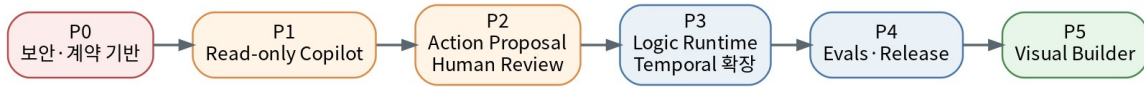


그림 6. 권장 구현 단계

14.1 P0. 보안·계약 기반

14.1.1 작업

- Object/Content search security partition 설계
- LanguageModelAdapter
- fake model adapter 와 contract test
- model/provider/alias repository
- AI run/event schema
- RetrievedContextItem, ToolSpec, ActionProposal
- AI-specific permission naming
- egress policy service

14.1.2 Exit Gate

- application layer 에서 provider SDK import 0 건
- unauthorized context provider 전달 0 건
- model/prompt/context/tool/policy hash 저장
- failure contract test 존재
- raw secret logging test 통과

14.2 P1. Read-only Copilot

14.2.1 작업

- published Agent Definition
- session/state store
- Object Context Provider
- Document Context Provider
- Retrieval Orchestrator
- Context Compiler
- Citation Service
- read-only ToolBroker
- chat API 와 minimal UI

14.2.2 Exit Gate

- exact source citation
- stale index hit 제거
- tenant/property leakage test 통과
- bounded loop
- model usage trace
- repeated request idempotency

14.3 P2. Action Proposal 과 Human Review

14.3.1 작업

- action.propose
- proposal fingerprint 와 expiry
- InsightReview schema 확장

- ApprovalExecutionService
- AI-review-action relation
- review UI 에서 source 와 diff 표시

14.3.2 Exit Gate

- 승인 없는 write 불가
- object version conflict 차단
- duplicate approval 이 duplicate action 을 만들지 않음
- proposal 변경 시 재승인
- external writeback compensation 보존

14.4 P3. Logic Runtime 와 Temporal 확장

14.4.1 작업

- typed DAG schema
- compiler/validator
- block runtime
- Temporal signal/cancel/query
- pause/resume approval
- schedule/event trigger

14.4.2 Exit Gate

- deterministic block replay
- bounded loop
- worker restart 후 resume
- workflow status 가 domain commit 을 대체하지 않음

14.5 P4. Evals 와 Release Governance

14.5.1 작업

- suite/case/run/result schema
- deterministic evaluators
- model/agent candidate comparison
- repeated-run variance
- release manifest
- canary/stable promotion 과 rollback

14.5.2 Exit Gate

- eval 미통과 version stable 승격 불가
- write agent security/action gate 필수
- previous stable pointer rollback 가능

14.6 P5. Visual Builder

Backend 계약이 안정된 후 구현한다.

- Agent Studio
- Context source editor
- Tool manifest editor
- Logic DAG canvas
- Eval dashboard
- AI run debugger

현재 웹은 단일 Operations HTML 중심이므로, Visual Builder 전에 frontend module/route/state architecture 를 정리해야 한다. [C-23]

14.7 권장 PR 순서

1. ai-contracts: domain DTO 와 ports 만 추가

2. ai-schema-ledger: migration/repository/contract tests
3. model-gateway-fake: fake adapter 와 deterministic tests
4. model-gateway-provider: 첫 provider-compatible adapter
5. retrieval-security: content/object security token contract
6. context-compiler: object/document providers 와 hashes
7. agent-runtime-readonly: bounded loop 와 citations
8. aip-api-sdk: API · generated SDK
9. action-proposal: proposal contract 와 UI
10. approval-execution: review-to-action bridge
11. ai-operations: run/event detail, usage, trace
12. logic-runtime: DAG and Temporal signals
13. aip-evals: suites and release gates
14. visual-builder: 마지막 단계

15 테스트·평가·릴리스 게이트

15.1 Contract tests

각 신규 port 에는 contract test 가 있어야 한다.

- LanguageModelAdapter
- ModelRegistryRepository
- AiDefinitionRepository
- AiRunRepository
- ContextProvider
- ToolExecutor
- UsageMeter

15.2 Deterministic security tests

```
cross_tenant_object_never_retrieved
masked_property_never_compiled
unauthorized_content_not_ranked_or_returned
provider_egress_denied_before_network_call
secret_value_never_logged
tool_not_in_agent_manifest_rejected
write_tool_requires_confirmation_or_review
proposal_fingerprint_change_requires_new_review
approval_rechecks_object_version
```

15.3 Retrieval eval

- golden query-source pairs
- Recall@K / MRR
- hybrid versus lexical/dense baseline
- chunk/version/hash integrity
- security filter false positive/negative
- token-budget selection quality

15.4 Tool and Action eval

- tool selection precision
- schema-valid arguments
- correct object/action target
- expected object version inclusion
- precondition compliance
- user confirmation behavior
- human review enforcement
- no direct write

15.5 Model/Prompt eval

- same case repeated N times
- output variance
- structured output pass rate
- unsupported claim rate
- citation faithfulness
- cost/latency
- candidate versus stable regression

15.6 CI quality gates

기존 package.json 의 세분화된 quality gate 스타일을 확장한다. [C-24]

```
quality:ai-contracts  
quality:model-gateway  
quality:ai-ledger  
quality:retrieval-security  
quality:context-compiler  
quality:tool-broker  
quality:action-proposal  
quality:approval-execution  
quality:ai-evals  
quality:ai-release
```

Release gate 는 static, unit, integration, live provider smoke 를 분리한다. 실제 provider smoke 는 credential 과 비용이 필요한 별도 lane 으로 둔다.

16 운영·관찰 가능성·비용 통제

16.1 핵심 대시보드

16.1.1 Runtime health

- runs by status
- model/tool error rate
- p50/p95 latency
- time to first token
- pending human reviews
- workflow wait duration

16.1.2 Retrieval quality

- zero-context rate
- citation coverage
- stale/hash mismatch
- security omissions count
- context tokens per turn

16.1.3 Tool safety

- denied tool calls
- schema-invalid calls
- confirmation rate
- proposal-to-approval rate
- approval-to-action success
- object version conflicts

16.1.4 Cost

- tokens/cost by tenant, agent, model alias
- cost per successful answer
- cost per approved action
- cache/retrieval savings
- candidate model comparison

16.2 Distributed trace

Trace span 예시:

```
POST /messages
ai.definition.resolve
ai.policy.evaluate
ai.state.snapshot
ai.retrieval.object
ai.retrieval.content
ai.context.compile
ai.model.complete
ai.tool.authorize
ai.tool.execute
ai.citation.resolve
ai.response.commit
```

기존 OpenTelemetry 기반과 correlation/request ID 를 그대로 연동한다.

16.3 Operational failure semantics

- model timeout: retryable, same run attempt chain

- egress denied: non-retryable policy failure
- context hash mismatch: fail closed
- tool unavailable: read-only answer 로 degrade 할지 agent policy 가 결정
- action version conflict: 새 proposal 필요
- review expired: 새 review 필요
- provider usage limit: retry-after 와 operator evidence
- start unknown: workflow ledger 의 기존 패턴 재사용

17 안티패턴과 최종 권고

17.1 반드시 피해야 할 구현

1. FastAPI route 가 provider SDK 를 직접 호출
2. LLM 이 만든 SQL 을 실행
3. repository·SQLAlchemy engine 을 Agent Runtime 에 주입
4. search index hit 를 DB 검증 없이 prompt 에 삽입
5. 모델이 만든 URL/citation 을 그대로 신뢰
6. service account 권한으로 사용자 tool 실행
7. mutable prompt/model alias 덮어쓰기
8. raw prompt 와 PII 를 일반 audit JSON 에 무기한 저장
9. vector index 를 source of truth 로 취급
10. generic HTTP/shell/python tool 제공
11. visual canvas 와 multi-agent 부터 시작
12. write-producing agent 의 silent model fallback
13. human approval 후 parameter 를 변경해 실행
14. workflow 성공 상태를 domain commit 의 성공으로 간주

17.2 최종 권고

최종 아키텍처 방향. foundry-lite 는 별도 AI 제품을 옆에 붙이는 대신, 현재의 Operational Object System 을 AI 가 안전하게 읽고 제안하고 실행하는 **Ontology-grounded AIP-lite layer** 로 확장해야 한다.

가장 큰 기존 자산은 다음과 같다.

1. Object 와 Action 이 typed operational boundary 다.
2. search index 를 source of truth 로 취급하지 않는다.
3. idempotency, audit, outbox, lineage 가 시스템 전반에 존재한다.
4. media/content unit 이 immutable version 과 citation coordinate 를 가진다.
5. Temporal 과 run ledger 가 durable operation 의 기반을 제공한다.

가장 먼저 담아야 할 공백은 다음과 같다.

1. 검색과 provider 반출의 security partition
2. Model Gateway 와 immutable Model Alias
3. AI Run/Event Ledger
4. Context Compiler
5. Tool Broker
6. review 승인과 Action 실행의 안전한 브리지

이 순서를 따르면 기존 설계를 대수술하지 않고 다음 상태에 도달할 수 있다.

Ontology-grounded, permission-aware, evidence-backed, action-capable AI operating layer

18 부록 A. Palantir 공식 소스 레지스터

ID	문서	URL	주요 사용 근거
P-01	AIP, Foundry, and Apollo	공식 문서	플랫폼 역할과 통합 구조
P-02	AIP architecture overview	공식 문서	AIP 12 개 capability 범주
P-03	The Ontology system	공식 문서	데이터·로직·액션·보안 통합
P-04	Chatbot Studio core concepts	공식 문서	prompt compilation, context window
P-05	Retrieval context	공식 문서	deterministic retrieval, context types
P-06	Application state	공식 문서	string/object-set state, visibility
P-07	Chatbot tools	공식 문서	Action, Object Query, Function, state tool
P-08	AIP Logic blocks	공식 문서	LLM/action/function/loop/condition blocks
P-09	Action types overview	공식 문서	Action transaction 과 side effect
P-10	Action permissions	공식 문서	visibility 와 submission criteria
P-11	Citations	공식 문서	object/document citation format
P-12	Session logging	공식 문서	structured events 와 trace ID
P-13	AIP observability	공식 문서	metrics, history, tracing, logs
P-14	Model Catalog	공식 문서	model discovery/comparison/playground
P-15	LLM-provider compatible APIs	공식 문서	proxy, governance, rate, usage
P-16	AIP Evals	공식 문서	test cases, models, variance
P-17	Apollo introduction	공식 문서	autonomous deployment/change management
P-18	Ontology augmented generation	공식 문서	hybrid search, HyDE, augmentation

19 부록 B. foundry-lite 코드 교차참조

ID	파일	검토 포인트	검토 당시 blob SHA
C-01	foundry_lite_development_plan_ko_sprintified.md	제품 페루프와 v1 경계	b76b6f8...
C-02	libs/foundry_lite/application/dependencies.py	CoreDependencies/ports	2116c80...
C-03	libs/foundry_lite/application/core_services.py	service graph/collaborators	d10341d...
C-04	libs/foundry_lite/application/foundry.py	root facade	677ce4d...
C-05	docs/implementation-status.md	실제 구현 경계	4ed6c33...
C-06	application/services/object_store/query.py	permission/masking/explain/query	317e105...
C-07	application/services/object_store/search.py	projection/re-read/mapping	30f5fa6...
C-08	application/services/object_store/semantic_search.py	model-pinned vector	2b5dfe3...
C-09	application/services/media_service.py	Media service group	a346d78...
C-10	infrastructure/schema.py	content units/media derivatives/security envelope	636b280...
C-11	application/services/media/retrieval.py	authoritative content re-read	3097624...
C-12	application/ports/embedding_model.py	embedding projection contract	c6c7784...
C-13	application/services/action_service.py	Action runtime/writeback/idempotency	c25973a...
C-14	application/safe_expression.py	parameter/precondition validation	1333110...
C-15	application/services/insight_review_service.py	human review/proposal	40221fa...
C-16	application/services/workflow_orchestration_service.py	workflow intent ledger	e886738...
C-17	application/ports/workflow_adapter.py	workflow port	0880e61...
C-18	infrastructure/adapters/temporal_workflow.py	Temporal idempotency/failures	f70c84d...
C-19	security/policy.py	RBAC/classification/masking	c16d82e...
C-20	application/services/runtime_service.py	audit/outbox/lineage/relation	d268c68...
C-21	application/ports/runtime_repository.py	runtime run types	3b74b6f...
C-22	application/services/ai_evidence.py	AI evidence/version/revision	ad630c8...
C-23	apps/web/index.html	현재 단일 Operations UI	342be85...
C-24	package.json	quality gates	8c9b68e...

20 부록 C. 제안 ADR 목록

1. ADR-AIP-001: LLM 은 untrusted planner 다.
2. ADR-AIP-002: AI runtime 은 repository 를 직접 호출하지 않는다.
3. ADR-AIP-003: 모든 write 는 Ontology Action 을 통과한다.
4. ADR-AIP-004: Retrieval index 는 rebuildable projection 이다.
5. ADR-AIP-005: Prompt/context/tool/model 은 immutable version 으로 pin 한다.
6. ADR-AIP-006: Search security 는 candidate pre-filter 를 요구한다.
7. ADR-AIP-007: High-risk write 는 exact proposal human approval 을 요구한다.
8. ADR-AIP-008: AI observability 는 structured event ledger 를 사용한다.
9. ADR-AIP-009: Raw prompts 는 별도 encrypted artifact 와 retention 을 사용한다.
10. ADR-AIP-010: Agent release 는 eval evidence 와 rollback pointer 를 요구한다.

21 부록 D. 용어집

용어	정의
Agent Definition	모델, prompt, context, tools, budget, policy 를 고정한 실행 정의
Model Alias	agent 가 참조하는 논리 모델 이름과 실제 provider revision 의 mapping
Context Compiler	권한 적용 context 와 tool/state 를 deterministic model request 로 조합하는 계층
Tool Broker	모델 tool call 을 authorization · schema · risk · budget 기준으로 증재하는 계층
Action Proposal	LLM 이 제안하지만 아직 실행되지 않은 정규화 write intent
AI Run Ledger	한 AI execution 의 상태와 version/hash/usage 를 보존하는 원장
Context Manifest	해당 model call 에 사용된 모든 context item identity 와 version 목록
Egress Policy	어떤 데이터 분류를 어떤 provider/region 으로 보낼 수 있는지 결정하는 정책
Authoritative re-read	index hit 를 진실로 신뢰하지 않고 원본 DB/object/content 를 재검증하는 절차
Bounded loop	model/tool/time/token/cost 상한이 있는 agent execution loop